

c0c2503878 / ProjExD_Group05

🔍

👤

+

🔄

🔗

📄

📁

🏠

<> Code

🔗 Pull requests

🎬 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

⚙ Settings

ProjExD_Group05 / README.md

...

🏠 c0c2503878 README更新

4c0ac87 · now

🔄

PreviewCodeBlame26 lines (20 loc) · 806 Bytes

Raw📄📥✎

丸焼き豪華豚

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1
- 必要なものがあれば追記してください（非推奨）

🔗 ゲームの概要

ゲームの遊び方

- マップを移動（矢印キー）して敵と戦う

ゲームの実装

共通基本機能

*背景画像と主人公キャラクターの描画

分担追加機能

- 戦闘イベント2パターン（担当：ハコザキ、カゲヤマ）
- ボス戦（担当：タドコロ）
- ミニゲーム（担当：ハヤシ）
- 罠（担当：タカハシ）

メモ

- 色は定数として定義して使う

- 定数、変数を定義するときは何に使用するか明確に示す
- classを定義するときは{イベント名}{class名}で定義する

c0c2503878 / ProjExD_Group05

<> Code

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

ProjExD_Group05 / Map_Base.py

c0c2503878 巽設置

aad7b94 · 27 minutes ago

Code

Blame

436 lines (392 loc) · 19.1 KB

Raw

```
1 import math
2 import os
3 import random
4 import sys
5 import time
6 import pygame as pg
7
8 os.chdir(os.path.dirname(os.path.abspath(__file__))) # この.pyファイルがあるフォルダをカレントディレクトリにする
9
10 # =====↓定数定義↓=====
11
12 WIDTH = 1240 # 横幅(x)
13 HEIGHT = 680 # 縦幅(y)
14 FPS = 60 # フレーム数
15 # マップのデータ（シード値）を格納しているもの（0：道、移動可能，1：障害物、移動不可，2：敵）
16 SEEDS = [
17     [ # 最上段
18         [ # マップ番号(0, 0) 左上
19             [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
20             [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
21             [1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
22             [1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
23             [1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
24             [1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0],
25             [1, 1, 0, 2, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1],
26             [1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 2, 0, 0, 1, 1, 1, 0, 1, 1, 1],
27             [1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
28             [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
29         ],
30         [ # マップ番号(0, 1) 真ん中上
31             [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
32             [1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1],
33             [1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1],
34             [1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 2, 0, 0, 0, 0, 1, 1, 0, 1, 1],
35             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0],
36             [0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
37             [1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1],
38             [1, 0, 0, 1, 1, 1, 0, 0, 0, 2, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1],
39             [1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1],
40             [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
41         ],
42     ],
43 ]
```

```
42     [ # マップ番号(0, 2) 右上
43         [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
44         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1],
45         [1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 2, 0, 1, 1, 1, 1, 1],
46         [1, 1, 1, 0, 0, 2, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1],
47         [0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1],
48         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1],
49         [1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 2, 1, 1, 0, 0, 1, 1],
50         [1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1],
51         [1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1],
52         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
53     ],
54 ],
55 [ # 中央段
56     [ # マップ番号(1, 0) 真ん中左
57         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
58         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
59         [1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1],
60         [1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1],
61         [1, 0, 0, 1, 1, 1, 2, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0],
62         [1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0],
63         [1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
64         [1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1],
65         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 2, 0, 1, 1],
66         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
67     ],
68     [ # 初期位置 マップ番号(1, 1) 中心
69         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
70         [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
71         [1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1],
72         [1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1],
73         [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
74         [0, 0, 5, 5, 5, 5, 5, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
75         [1, 0, 1, 1, 5, 5, 5, 5, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1],
76         [1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1],
77         [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
78         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
79     ],
80     [ # マップ番号(1, 2) 真ん中右
81         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
82         [1, 1, 1, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
83         [1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1],
84         [1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1],
85         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 2, 1, 1, 1],
86         [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1],
87         [1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1],
88         [1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 2, 0, 0, 0, 1, 1],
89         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1],
90         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
91     ],
92 ],
93 [ # 最下段
94     [ # マップ番号(2, 0) 左下
95         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
96         [1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
97         [1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1],
98         [1, 1, 1, 2, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1],
99         [1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0],
```

```

100         [1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0],
101         [1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1],
102         [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 2, 1, 1, 1, 1, 1, 1],
103         [1, 1, 1, 1, 1, 1, 2, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
104         [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
105     ],
106     [ # マップ番号(2, 1) 真ん中下
107         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
108         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
109         [1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1],
110         [1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1],
111         [0, 0, 2, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0],
112         [0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0],
113         [1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1],
114         [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1, 1, 1, 1, 1, 1],
115         [1, 1, 1, 1, 1, 1, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
116         [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
117     ],
118     [ # マップ番号(2, 2) 右下
119         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
120         [1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 1, 1, 1],
121         [1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1],
122         [1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1],
123         [0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1],
124         [0, 2, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
125         [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
126         [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 2, 1, 1, 1, 1, 1],
127         [1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1],
128         [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
129     ],
130 ],
131 ]
132
133 # 色
134 # ===↓色定義↓===
135 BLACK = (0, 0, 0)
136 WHITE = (255, 255, 255)
137 RED = (255, 0, 0)
138 GREEN = (0, 255, 0)
139 BLUE = (0, 0, 255)
140 PURPLE = (255, 0, 255)
141 # ===↑色定義↑===
142
143 # =====↑定数定義↑=====
144
145
146 # ===↓class定義↓===
147
148 class GameMap:
149     """
150     マップを作成するもの
151     """
152     def __init__(self, y_num: int, x_num: int):
153         """
154         マップを作成し、データを初期化する
155         引数：縦の分割数int, 横の分割数int
156         y_numはHEIGHTを割り切れる値が好ましい, x_numはWIDTHを割り切れる値が好ましい
157         """

```

```
158
159     self.x_num = x_num
160     self.y_num = y_num
161     self.wid = WIDTH // x_num
162     self.hei = HEIGHT // y_num
163     self.map_data = self._create_map_data()
164     self.rocks = pg.sprite.Group() # 岩のグループ作成
165     self.traps = pg.sprite.Group() # 罌のグループ作成
166
167     def _create_map_data(self) -> list[list[dict]]:
168         """
169         二次元リストを作成する隠しメソッド
170         戻り値：二次元リストマップデータ (list[[データ]], アクセスlist[縦][横][データ])
171         """
172
173         map_data = [] # 大元の二次元リストになるもの
174         for row in range(self.y_num):
175             row_lis = [] # そのマスのデータを格納するもの
176             for col in range(self.x_num):
177                 cell_data = {
178                     # マスの番号
179                     "id": (row, col),
180                     # マスの中心座標 (オブジェクト用)
181                     "coor": (self.wid * col + (self.wid // 2), self.hei * row + (self.hei // 2)),
182                     # マスのステータス (0: 移動可能, 1: 移動不可, 2: 敵とのバトルイベント)
183                     "type": 0
184                 }
185                 row_lis.append(cell_data)
186             map_data.append(row_lis)
187         return map_data
188
189     def update_line(self, screen: pg.Surface):
190         """
191         マップに線を描画するもの
192         デバッグ用
193         """
194         for i in range(1, self.x_num):
195             pg.draw.line(screen, RED, (self.wid * i, 0), (self.wid * i, HEIGHT), 1)
196         for i in range(1, self.y_num):
197             pg.draw.line(screen, RED, (0, self.hei * i), (WIDTH, self.hei * i), 1)
198
199     def get_cell(self, row: int, col: int) -> dict:
200         """
201         指定した座標のデータを取得するもの
202         引数：縦番号int, 横番号int
203         戻り値：マスの情報 (データ) , マスが存在しない場合None
204         """
205         if 0 <= col < self.x_num and 0 <= row < self.y_num:
206             return self.map_data[row][col]
207         return None
208
209     def load_map(self, map_y: int, map_x: int):
210         """
211         マップを生成するもの
212         引数：読み込みたいマップの番号map_y(0, 1, 2), map_x(0, 1, 2)
213         """
214         self.rocks.empty() # 前のマップの岩を全て削除
215         self.traps.empty() # 前のマップの罌を消去
```

```

216 seed = SEEDS[map_y][map_x] # 指定された座標のマップのデータを取得
217 for row in range(self.y_num):
218     for col in range(self.x_num):
219         if seed[row][col] == 1: # 岩のマスか判定
220             rock = Rock(self.map_data[row][col]["coor"]) # 岩を生成
221             self.rock.add(rock) # 岩を岩グループに追加
222         elif seed[row][col] == 5: # 5だったら罠を作る
223             trap = Trap(self.map_data[row][col]["coor"]) # 罠生成
224             self.traps.add(trap) # 罠を罠グループに追加
225             self.map_data[row][col]["type"] = seed[row][col] # seedに沿ってtypeを上書 (マップ形成)
226
227 ✓ def check_move(self, row: int, col: int) -> int:
228     """
229     移動できるのかを判定するもの
230     引数: 移動先の縦番号int, 移動先の横番号int
231     戻り値: 移動先のマスの"type"int (0: 移動可能, 1: 移動不可, 2: 敵)
232     """
233     cell = self.get_cell(row, col) # 移動先のマスのデータを取得
234     if cell is None:
235         return 1 # マップ外は壁扱いにする
236     return cell["type"] # 移動先のマスのtypeを返す
237
238 ✓ def get_enemy_positions(self) -> list[tuple[int, int]]:
239     """
240     敵が配置されているマスの中心座標を取得する
241     戻り値: 敵が配置されているマスの中心座標tupleが格納されたlist
242     """
243     positions = [] # 敵がいるマスの中心座標を格納するもの
244     for row in range(self.y_num):
245         for col in range(self.x_num):
246             if self.map_data[row][col]["type"] == 2:
247                 positions.append(self.map_data[row][col]["coor"])
248     return positions
249
250 ✓ def get_id(self, x: int, y: int) -> tuple[int, int]:
251     """
252     座標からマス目idを求めるもの
253     引数: オブジェクトの中心のx座標, y座標
254     戻り値: tuple(行, 列)
255     """
256     col = x // self.wid
257     row = y // self.hei
258     return row, col
259
260 ✓ def update(self, screen: pg.Surface):
261     """
262     画面に岩などを描画するもの
263     引数: 画像Surface
264     """
265     self.rock.draw(screen)
266     self.traps.draw(screen)
267
268
269 ✓ class Rock(pg.sprite.Sprite):
270     """
271     岩に関するもの
272     """
273 ✓ def __init__(self, coor: tuple[int, int]):

```

```
274     """
275     引数：マスを中心座標tuple(x, y)
276     """
277     super().__init__()
278     self.image = pg.Surface((55, 55)) # 現時点仮の岩画像
279     self.image.fill(BLACK) # 現時点仮の岩
280     self.rect = self.image.get_rect(center = coor) # rect.centerにcoorを設定
281
282
283     class Enemy(pg.sprite.Sprite):
284         """
285         敵に関するもの
286         """
287
288     def __init__(self, coor: tuple[int, int]):
289         """
290         引数：マスを中心座標tuple(x, y)
291         """
292         super().__init__()
293         self.image = pg.Surface((40, 40)) # 現時点仮の敵画像
294         self.image.fill(RED) # 現時点仮の敵
295         self.rect = self.image.get_rect(center = coor) # rect.centerにcoorを設定
296
297
298     class Player(pg.sprite.Sprite):
299         """
300         プレイヤーに関するもの
301         """
302     def __init__(self, coor: tuple[int, int], game_map: GameMap):
303         """
304         引数：初期配置の中心座標tuple(x, y), GameMapのインスタンス
305         """
306         super().__init__()
307         self.image = pg.Surface((40, 40)) # 現時点仮のプレイヤー画像
308         self.image.fill(GREEN) # 現時点仮のプレイヤー
309         self.rect = self.image.get_rect(center = coor) # rect.centerにcoorを設定
310         self.game_map = game_map
311         self.row, self.col = self.game_map.get_id(coor[0], coor[1]) # プレイヤーのいるマスのidを取得
312
313     def move(self, move_row: int, move_col: int) -> str | None:
314         """
315         指定された方向へ1マス移動する(idを参照して移動)
316         引数：縦の移動量, 横の移動量
317         戻り値：マップ外に出た場合は外に出た方向の文字列、それ以外はNone
318         """
319         next_row = self.row + move_row
320         next_col = self.col + move_col
321
322         # 画面外への移動判定 (マップ移動)
323         if next_row < 0:
324             return "UP"
325         if next_row >= self.game_map.y_num:
326             return "DOWN"
327         if next_col < 0:
328             return "LEFT"
329         if next_col >= self.game_map.x_num:
330             return "RIGHT"
331         # 移動先の判定 (岩ではない場合)
```



```
332         if self.game_map.check_move(next_row, next_col) != 1:
333             self.row = next_row
334             self.col = next_col
335             self.rect.center = self.game_map.get_cell(self.row, self.col)["coor"]
336         return None
337
338     class Trap(pg.sprite.Sprite):
339         """
340         罌に関するもの（見える罌）
341         今回は毒
342         """
343     def __init__(self, coor: tuple[int, int]):
344         super().__init__()
345         self.image = pg.Surface((30, 30)) # プレイヤーより少し小さめのサイズ
346         self.image.fill(PURPLE) # 紫で見えるようにする
347         self.rect = self.image.get_rect(center=coor)
348
349     # ===↑class定義↑===
350
351
352     # ===↓関数定義↓===
353
354
355     # ===↑関数定義↑===
356
357
358     def main():
359         pg.display.set_caption("ゲーム(仮)")
360
361         # ===↓変数定義↓===
362         screen = pg.display.set_mode((WIDTH, HEIGHT))
363         clock = pg.time.Clock()
364         game_map = GameMap(10, 20) # マップ作成
365         current_map_x = 1 # マップ初期x座標
366         current_map_y = 1 # マップ初期y座標
367         # マップ番号(1, 1) 中心
368         game_map.load_map(current_map_y, current_map_x) # マップロード
369         enemys = pg.sprite.Group() # 敵のグループ作成
370         # 敵の座標読み込み
371         for coor in game_map.get_enemy_positions():
372             enemys.add Enemy(coor)
373         start_coor = game_map.get_cell(5, 10)["coor"] # 初期位置
374         player = Player(start_coor, game_map) # プレイヤー定義
375         players = pg.sprite.GroupSingle(player) # プレイヤー用グループ (単体)
376         # ===↑変数定義↑===
377
378         while True:
379             for event in pg.event.get():
380                 if event.type == pg.QUIT: return # 終了判定
381
382                 # プレイヤー移動処理
383                 if event.type == pg.KEYDOWN:
384                     move: str | None = None # マップ移動の詳細を格納する変数
385                     if event.key == pg.K_UP:
386                         move = player.move(-1, 0)
387                     elif event.key == pg.K_DOWN:
388                         move = player.move(1, 0)
389                     elif event.key == pg.K_LEFT:
```

```
390         move = player.move(0, -1)
391     elif event.key == pg.K_RIGHT:
392         move = player.move(0, 1)
393     # マップ移動処理
394     if move:
395         if move == "UP" and current_map_y > 0:
396             current_map_y -= 1
397             player.row = game_map.y_num - 1 # 下端
398         elif move == "DOWN" and current_map_y < 2:
399             current_map_y += 1
400             player.row = 0 # 上端
401         elif move == "LEFT" and current_map_x > 0:
402             current_map_x -= 1
403             player.col = game_map.x_num - 1 # 右端
404         elif move == "RIGHT" and current_map_x < 2:
405             current_map_x += 1
406             player.col = 0 # 左端
407     else:
408         continue
409     # 新しいマップをロード
410     game_map.load_map(current_map_y, current_map_x)
411     enemys.empty() # 移動前のマップに表示されている敵を削除
412     # 敵の座標読み込み
413     for coor in game_map.get_enemy_positions():
414         enemys.add Enemy(coor)
415     # プレイヤーを更新
416     player.rect.center = game_map.get_cell(player.row, player.col)["coor"]
417     if game_map.check_move(player.row, player.col) == 2: # 移動した先が敵かの判定
418         pass # ここにバトルイベントなどを追加
419
420     screen.fill(WHITE) # 背景仮(一番最初に描画)
421     game_map.update_line(screen) # 枠線表示 (デバッグ用)
422
423     game_map.update(screen) # 岩を描画
424     enemys.draw(screen) # 敵を描画
425     players.draw(screen) # プレイヤーを描画
426
427
428     pg.display.update()
429     clock.tick(FPS)
430
431
432 if __name__ == "__main__":
433     pg.init()
434     main()
435     pg.quit()
436     sys.exit()
```

c0c2503878 / ProjExD_Group05

<> Code

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Comparing changes

Choose two branches to see what’s changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main

compare: C0C25038/巛設置

✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#)

Create pull request

2 commits

2 files changed

1 contributor

Commits on Jul 7, 2026

巛設置

c0c2503878 committed 27 minutes ago

aad7b94

README更新

c0c2503878 committed 1 minute ago

4c0ac87

Showing 2 changed files with 30 additions and 13 deletions.

Split Unified

22 Map_Base.py

71	71		[1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1],
72	72		[1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1],
73	73		[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
74		-	[0, 0],
75		-	[1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1],
	74	+	[0, 0, 5, 5, 5, 5, 5, 5, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
	75	+	[1, 0, 1, 1, 5, 5, 5, 5, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1],
76	76		[1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1],
77	77		[1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
78	78		[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
137	137		RED = (255, 0, 0)
138	138		GREEN = (0, 255, 0)
139	139		BLUE = (0, 0, 255)
	140	+	PURPLE = (255, 0, 255)
140	141		# ===↑色定義↑===
141	142		
142	143		# =====↑定数定義↑=====
161	162		self.hei = HEIGHT // y_num

162	163	self.map_data = self._create_map_data()
163	164	self.rock = pg.sprite.Group() # 岩のグループ作成
	165	+ self.traps = pg.sprite.Group() # 罨のグループ作成
164	166	
165	167	def _create_map_data(self) -> list[list[dict]]:
166	168	"""
210	212	引数: 読み込みたいマップの番号map_y(0, 1, 2), map_x(0, 1, 2)
211	213	"""
212	214	self.rock.empty() # 前のマップの岩を全て削除
	215	+ self.traps.empty() # 前のマップの罨を消去
213	216	seed = SEEDS[map_y][map_x] # 指定された座標のマップのデータを取得
214	217	for row in range(self.y_num):
215	218	for col in range(self.x_num):
216	219	if seed[row][col] == 1: # 岩のマスか判定
217	220	rock = Rock(self.map_data[row][col]["coor"]) # 岩を生成
218	221	self.rock.add(rock) # 岩を岩グループに追加
	222	+ elif seed[row][col] == 5: # 5だったら罨を作る
	223	+ trap = Trap(self.map_data[row][col]["coor"]) #罨生成
	224	+ self.traps.add(trap) #罨を罨グループに追加
219	225	self.map_data[row][col]["type"] = seed[row][col] # seedに沿ってtypeを上書 (マップ形成)
220	226	
221	227	def check_move(self, row: int, col: int) -> int:
257	263	引数: 画像Surface
258	264	"""
259	265	self.rock.draw(screen)
	266	+ self.traps.draw(screen)
260	267	
261	268	
262	269	class Rock(pg.sprite.Sprite):
328	335	self.rect.center = self.game_map.get_cell(self.row, self.col)["coor"]
329	336	return None
330	337	
	338	+ class Trap(pg.sprite.Sprite):
	339	+ """
	340	+ 罨に関するもの (見える罨)
	341	+ 今回は毒
	342	+ """
	343	+ def __init__(self, coor: tuple[int, int]):
	344	+ super().__init__()
	345	+ self.image = pg.Surface((30, 30)) # プレイヤーより少し小さめのサイズ
	346	+ self.image.fill(PURPLE) # 紫で見えるようにする
	347	+ self.rect = self.image.get_rect(center=coor)
	348	+
331	349	# ===↑class定義↑===
332	350	
333	351	

▼ 21 README.md

6	6	* 必要なものがあれば追記してください (非推奨)
7	7	
8	8	## ゲームの概要
9		- * 主人公キャラクター豪華豚 (ゴウカトン) をマウス操作により丸焼きにするゲームで、...
10		- * 参考URL: [サイトタイトル](https://www.hoge.com/)
11	9	
12		- ## ゲームの遊び方
13		- * 矢印キーで豪華豚を操作し、スペースキー押下によるバーナーで豪華豚を丸焼きにする

14		- * 豪華豚が10秒以上バーナーにあぶられたら、ゲームオーバーとなる
15	10	
	11	+ ## ゲームの遊び方
	12	+ * マップを移動（矢印キー）して敵と戦う
16	13	## ゲームの実装
17	14	### 共通基本機能
18		- * 背景画像と主人公キャラクターの描画
	15	+ * 背景画像と主人公キャラクターの描画
19	16	
20	17	### 分担追加機能
21		- * 丸焼きエフェクト（担当：ふしみ）：バーナーにより豪華豚を丸焼きにするエフェクトに関するクラス
22		- * キッチンタイマー機能（担当：ぷしみ）：制限時間以内に調理が完了しなかった場合に、豪華豚が脱走する機能
23		- * 調理機能（担当：ぷしみ）：調理器具をキー押下により選択し、豪華豚を調理する機能
	18	+ * 戦闘イベント2パターン（担当：ハコザキ、カゲヤマ）
	19	+ * ボス戦（担当：タドコロ）
	20	+ * ミニゲーム（担当：ハヤシ）
	21	+ * 罨（担当：タカハシ）
24	22	
25	23	### メモ
26		- * クラス内の変数は、すべて、「get_変数名」という名前のメソッドを介してアクセスするように設計してある
27		- * すべてのクラスに関する関数は、クラスの外で定義してある
	24	+ * 色は定数として定義して使う
	25	+ * 定数、変数を定義するときは何に使用するか明確に示す
	26	+ * classを定義するときは{イベント名}{class名}で定義する